EC7207: High Performance Computing

Analysis Report

Parallel Pearson Correlation Based Recommendation System

Group Number: 11

EG/2021/4417 Ashfaq M.R.M.

EG/2021/4419 Athanayaka K.A.L.G.

EG/2021/4424 Balasooriya J.M.

Technologies: Serial (C) · OpenMP · Pthreads · MPI · CUDA · Hybrid MPI+OpenMP

Problem: 1 000 users × 1 000 items · SEED=42 · Sparsity=70% · TOP-K=20

Platform: Pop!_OS 24.04 LTS · GCC 13.3.0 · Open MPI 4.1.6 · CUDA 12.0

Intel Core i7-11800H (8 physical / 16 logical cores) · NVIDIA GeForce RTX 3050 Laptop GPU

May 2026

Contents

1 Introduction

Modern collaborative filtering recommendation systems compute pairwise user similarity over large rating matrices. For $N = M = 1{,}000$ users and items, this amounts to 499 500 Pearson pair computations totalling approximately 500 million floating-point operations — the dominant bottleneck.

This report documents the implementation and performance evaluation of a **Pearson Correlation-Based Collaborative Filtering Recommender System** parallelised with OpenMP, POSIX Threads (Pthreads), MPI, CUDA, and Hybrid MPI+OpenMP. All versions are benchmarked on a single Linux machine (Intel Core i7-11800H, 8 physical / 16 logical cores) with an NVIDIA GeForce RTX 3050 Laptop GPU for the CUDA version (see Section 4.7).

Experimental Configuration

Parameter	Value
Users (N)	1 000
Items (M)	1 000
Sparsity	70%
Test split	10% (29 866 held-out ratings)
TOP-K neighbours	20
Random seed	42
Benchmark thread/process counts	1, 2, 4, 8
Timing method	`clock_gettime` / `omp_get_wtime` / `MPI_Wtime` / `cudaEvent` (GPU)
Platform	Pop!_OS 24.04 LTS, GCC 13.3.0, Open MPI 4.1.6, CUDA 12.0
CPU / GPU	Intel Core i7-11800H (8 phys / 16 logical) / NVIDIA RTX 3050 Laptop GPU

Table 1: Fixed benchmark parameters used across all versions.

2 Algorithm Design and Parallel Programming Concepts

2.1 Algorithm Pipeline (5 Phases)

Phase	Name	Description	Complexity
1	Data Generation	Synthetic sparse matrix; 10% test split	$O(N \times M)$
2	User Means	Per-user mean μ_u	$O(N \times M)$
3	Similarity	Pearson for all (u, v) pairs — bottleneck	$O(N^2 \times M)$
4	Predictions	TOP-K weighted prediction	$O(N^2 \times M)$
5	MAE Evaluation	Error on held-out test set	$O(\|T\|)$

Table 2: Algorithm phases and computational complexity.

Pearson correlation (Phase 3):

$$\text{sim}(u, v) = \frac{\sum_i (R_{ui} - \mu_u)(R_{vi} - \mu_v)}{\sqrt{\sum_i (R_{ui} - \mu_u)^2}\ \sqrt{\sum_i (R_{vi} - \mu_v)^2}}$$

Prediction formula (Phase 4):

$$\hat{r}_{ui} = \mu_u + \frac{\sum_{k \in N_K(u)} \text{sim}(u, k)\,(R_{ki} - \mu_k)}{\sum_{k \in N_K(u)} \text{sim}(u, k)}$$

Both phases are *embarrassingly parallel at the row level*, making them ideal targets for all five parallelisation strategies. Figure 1 summarises the pipeline and marks which phases each technology parallelises; the row-wise decomposition that makes every model race-free is then shown in the per-technology diagrams that follow.

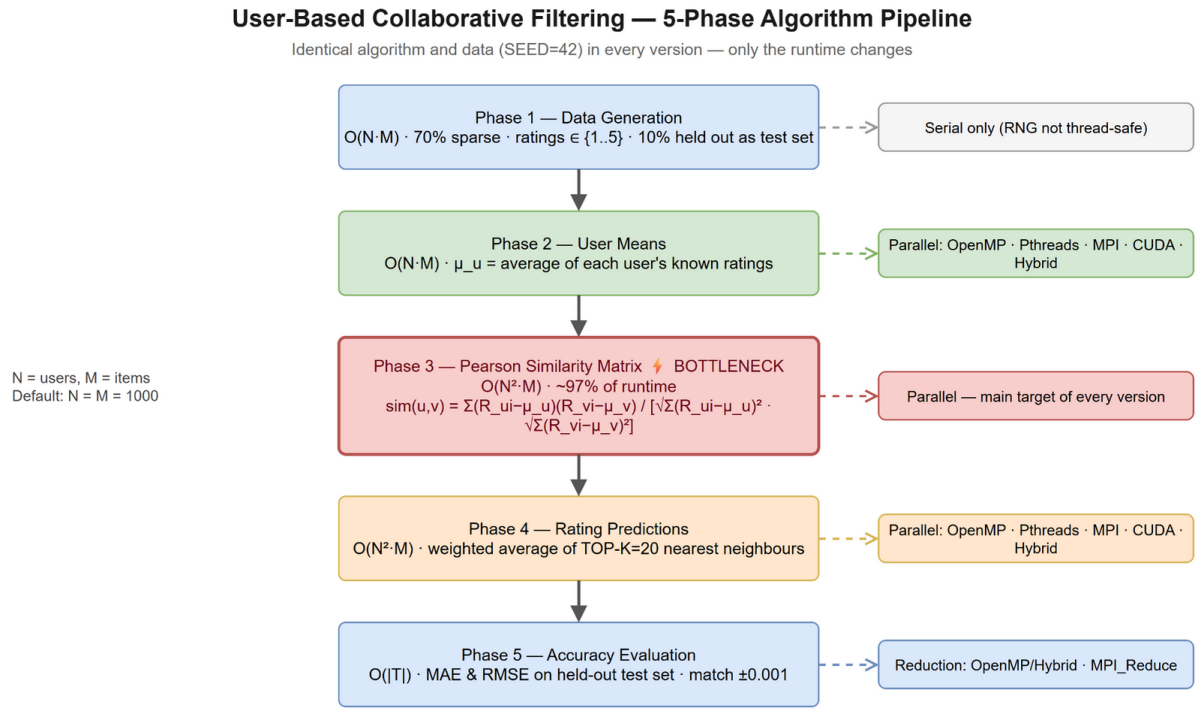

Figure 1: The five-phase recommender pipeline, shared by every implementation. The same algorithm and data (`SEED=42`) are used throughout — only the runtime changes. Phase 3 (Pearson similarity) is the $O(N^2M)$ bottleneck; Phases 3–4 are the targets of all five parallel models, while data generation stays serial.

2.2 OpenMP — Shared-Memory Fork-Join Parallelism

OpenMP exploits shared-memory parallelism using compiler directives. A master thread forks into T workers sharing the same address space, then rejoins at an implicit barrier (Figure 2).

Phase 3 (key design choice): The outer loop uses `#pragma omp parallel for schedule(dynamic,4)`. Dynamic scheduling is essential because the triangular loop creates severe load imbalance: thread 0 processes $N-1 = 999$ Pearson calls while the last thread processes 1. Dynamic scheduling lets idle threads pull 4-row chunks from a shared work queue, eliminating wasted CPU time.

Race-freedom: Each thread owns an exclusive set of outer-loop rows u and writes `SIM(u,v)` only for those rows. No two threads write the same cell; no mutex is required.

Phase 5 (MAE/RMSE): The `reduction(+:err)` clause gives each thread a private accumulator; OpenMP sums all copies at the barrier. The same pattern (`reduction(+:sq)`) accumulates squared error for RMSE.

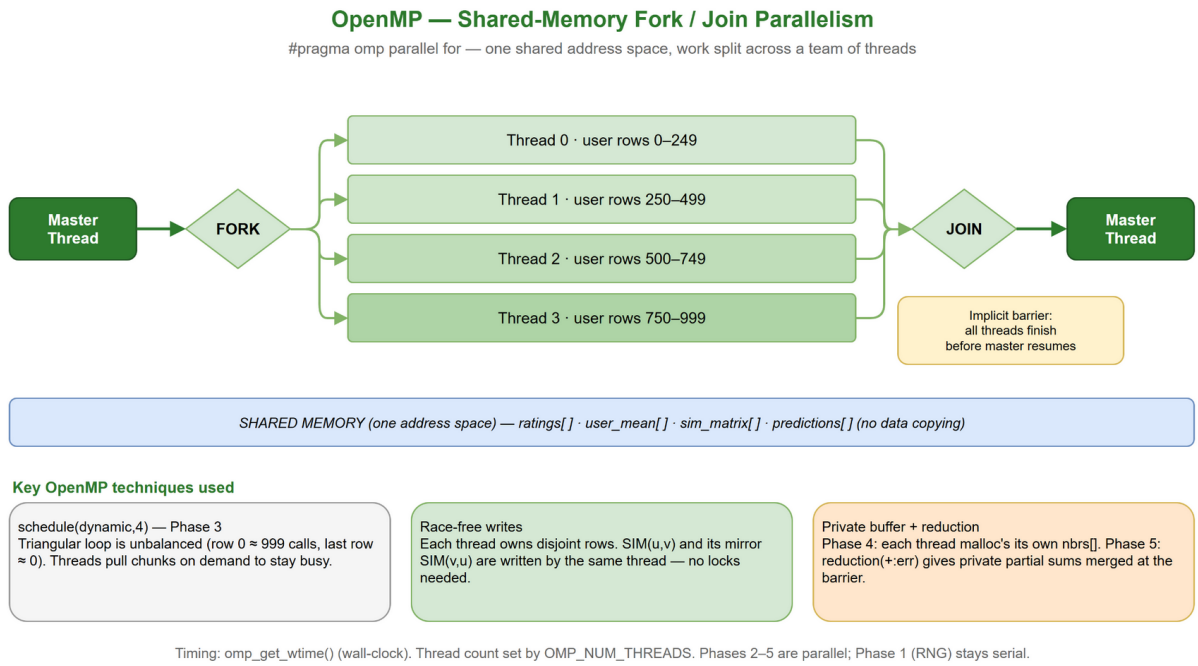

Figure 2: OpenMP fork/join model. The master thread forks a team of T workers over one shared address space (`ratings`, `sim_matrix`, ...), each owning a disjoint slice of user rows, then rejoins at an implicit barrier. The side notes show the three race-free techniques used: `schedule(dynamic,4)` for the triangular loop, disjoint-row writes, and a thread-private `nbrs[]` buffer with `reduction(+:err)`.

2.3 Pthreads — Manual POSIX Thread Management

POSIX Threads require the programmer to explicitly create threads, assign work, and synchronise with `pthread_join` (Figure 3).

A `run_parallel(fn, nthreads)` harness is called at the start of each of Phases 2, 3, and 4:

1. `pthread_create()` creates T threads, each receiving a `ThreadArgs` struct with its `tid` and total `nthreads`.
2. Each thread derives its exclusive row range: start $= tid \times \lceil N/T \rceil$, end $= \min(\text{start} + \lceil N/T \rceil, N)$.
3. `pthread_join()` acts as a phase barrier.

Key difference from OpenMP: Threads are created and joined three separate times (once per phase), paying OS thread-creation overhead three times per run. Static row assignment also cannot compensate for the triangular loop imbalance — which explains the slightly lower efficiency vs. OpenMP.

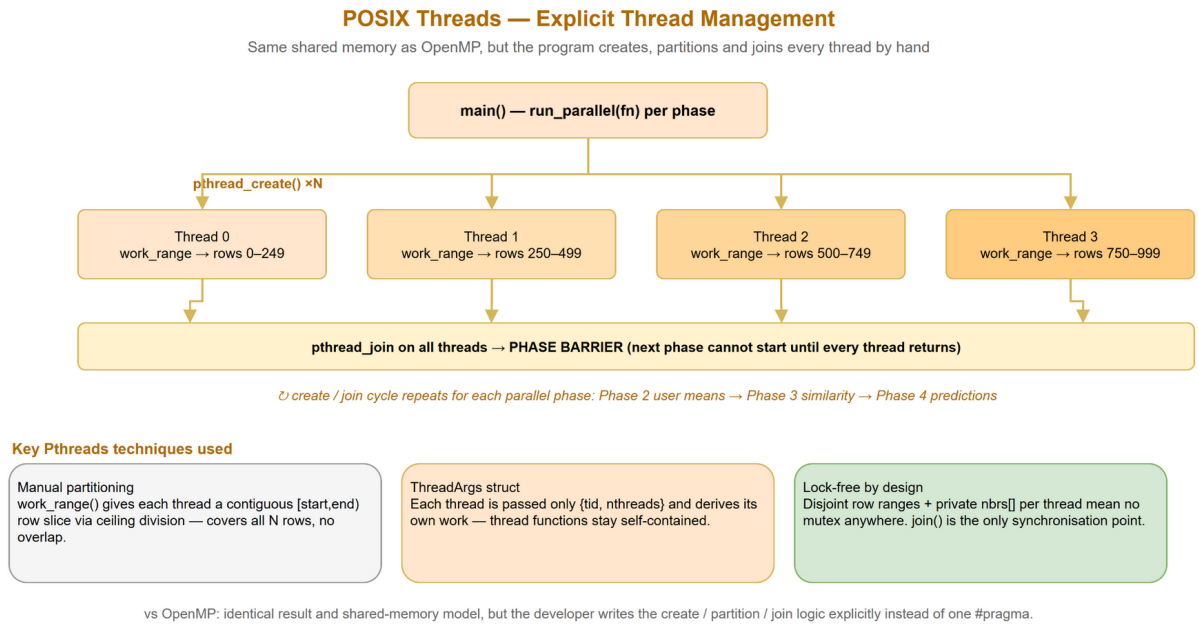

Figure 3: Manual POSIX-thread management. `main()` calls `run_parallel` once per parallel phase: `pthread_create` spawns T threads, each deriving its own `ceil`-division row range from a `ThreadArgs` struct, and `pthread_join` on all threads acts as the phase barrier. Disjoint rows plus a private `nbrs[]` buffer make it lock-free.

2.4 MPI — Distributed-Memory Parallelism

Each MPI rank is an independent process with its own private address space (Figure 4). The $N = 1{,}000$ users are divided evenly: rank r owns rows $[r \cdot N/P,\ (r{+}1) \cdot N/P)$.

Phase	Strategy	Primitive
Phase 1	All ranks call `srand(42)` — no communication	—
Phase 2	Each rank computes its `user_mean[]` slice	`MPI_Allgatherv`
Phase 3	Each rank fills its rows of the sim matrix	`MPI_Allgatherv`
Phase 4	Each rank predicts its own users	—
Phase 5	Local error sums to rank 0	`MPI_Reduce(SUM)`

Table 3: MPI communication strategy per phase.

The `MPI_Allgatherv` for the $N \times N$ similarity matrix transfers $\approx 4\,\text{MB}$ at $N = 1{,}000$. This overhead limits single-node efficiency but is the necessary mechanism for multi-node deployments.

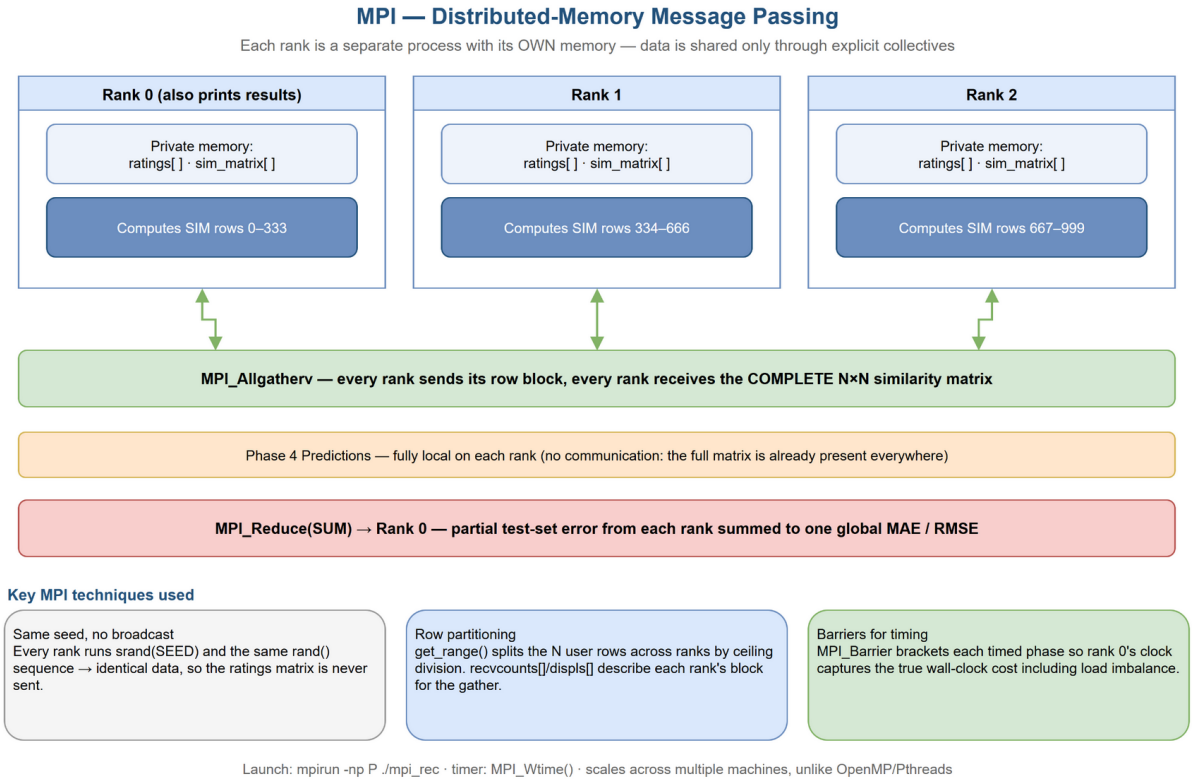

Figure 4: MPI distributed-memory layout. Each rank is a separate process with a private copy of the matrices, computing the similarity rows it owns. A single `MPI_Allgatherv` then gives every rank the complete $N \times N$ matrix (so predictions need no communication), and `MPI_Reduce` sums the partial test-set error to rank 0 for the global MAE/RMSE.

2.5 CUDA — GPU Massively Parallel Computing

Note: CUDA code (`cuda/cuda_recommender.cu`) is fully implemented and benchmarked on an NVIDIA GeForce RTX 3050 Laptop GPU (Ampere, compute capability 8.6, 16 SMs); measured results are reported in Section 4.7.

Kernel	Launch config	Assignment
`kernel_user_means`	`<<<(N+255)/256, 256>>>`	1 thread = 1 user
`kernel_similarity`	`dim3(⌈N/16⌉, ⌈N/16⌉), dim3(16,16)`	1 thread = 1 (u, v) pair
`kernel_predictions`	2D grid, users × items	1 thread = 1 (u, i) pair

Table 4: CUDA kernel configuration for $N = 1{,}000$.

The similarity kernel launches $63 \times 63 \times 256 = 1{,}016{,}064$ threads simultaneously (Figure 5). Timing uses `cudaEventRecord` to separate kernel-only time from `cudaMemcpy` transfer overhead.

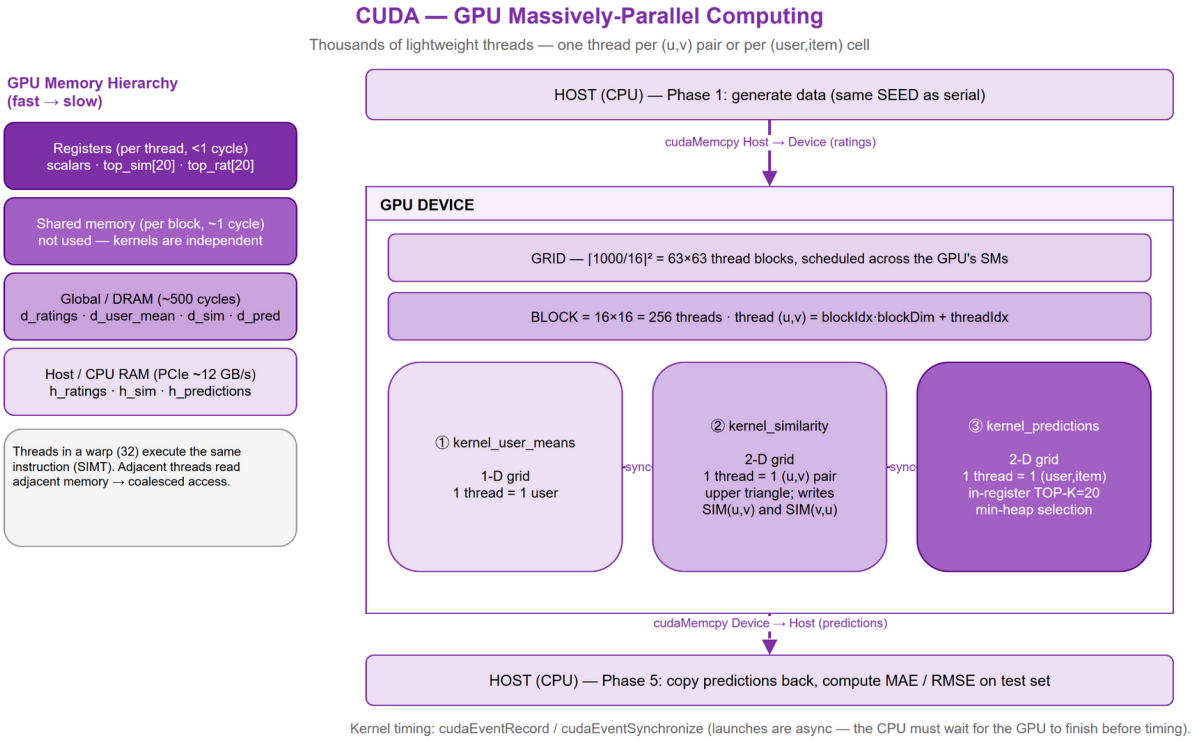

Figure 5: CUDA GPU model. After a Host→Device copy of the ratings, three kernels run in sequence on a 2D grid of 16×16 thread blocks — one thread per (u, v) pair / (user, item) cell — before the predictions are copied back. The left column shows the GPU memory hierarchy (registers → shared → global → host). Benchmarked on an NVIDIA RTX 3050 (compute capability 8.6).

2.6 Hybrid MPI+OpenMP — Two-Level Parallelism

Combines MPI (coarse-grain inter-node) and OpenMP (fine-grain intra-node), as shown in Figure 6. With P MPI ranks and T OpenMP threads per rank, the total worker count is $P \times T$ while sending only P messages per collective call.

`MPI_Init_thread(MPI_THREAD_FUNNELED)` ensures all MPI calls are made exclusively from the main thread, after OpenMP parallel regions complete.

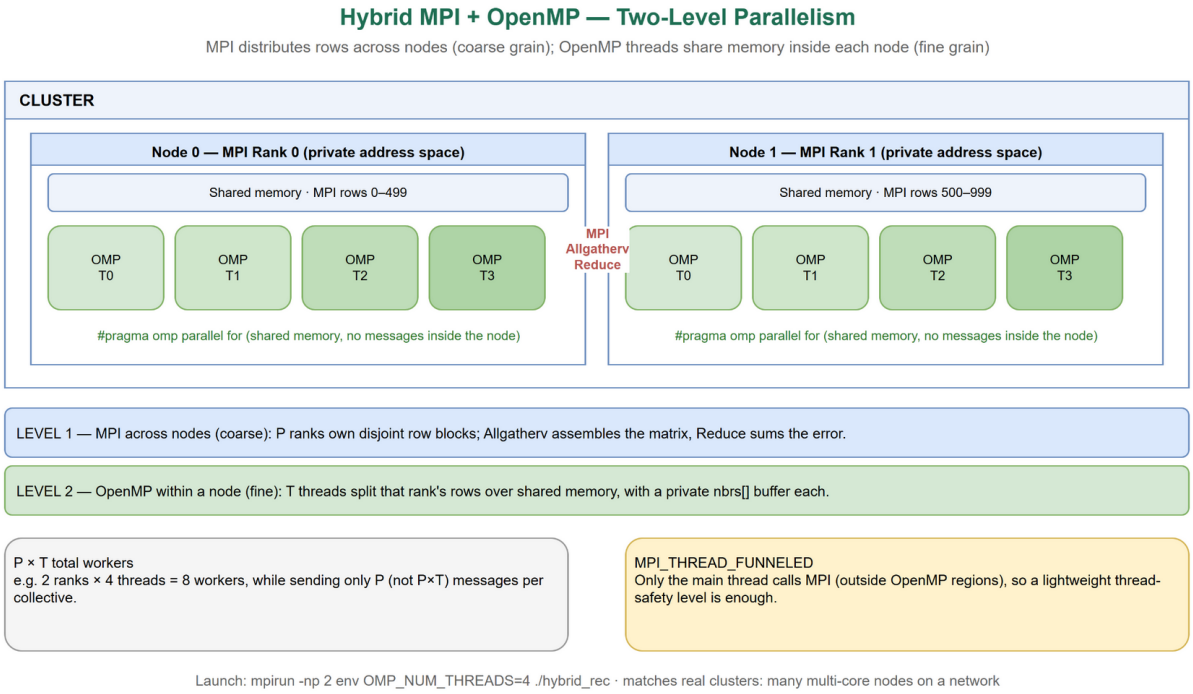

Figure 6: Hybrid two-level parallelism. Level 1 (MPI) partitions user-rows across P ranks on separate nodes; level 2 (OpenMP) spawns T threads over each rank's rows in shared memory, for $P \times T$ total workers. `MPI_Allgatherv` / `MPI_Reduce` link the ranks, and `MPI_THREAD_FUNNELED` keeps all MPI calls on the main thread.

Config	P	T	Characteristic
Hybrid 2×4	2	4	Fewer MPI messages; more shared-memory parallelism
Hybrid 4×2	4	2	Balanced communication vs. threading overhead
Hybrid 8×1	8	1	Equivalent to pure MPI
Hybrid 1×8	1	8	Pure OpenMP wrapped with MPI overhead

Table 5: Hybrid configurations (all 8 total workers).

3 Accuracy Analysis — Parallel vs. Serial

All parallel implementations are validated against the serial baseline using two error metrics — **Mean Absolute Error (MAE)** and **Root Mean Square Error (RMSE)** — together with a **similarity-matrix checksum**:

$$\text{MAE} = \frac{1}{|T|} \sum_{(u,i)\in T} |\hat{r}_{ui} - r_{ui}|, \qquad \text{RMSE} = \sqrt{\frac{1}{|T|} \sum_{(u,i)\in T} (\hat{r}_{ui} - r_{ui})^2} \qquad (|T| = 29{,}866)$$

MAE vs. RMSE. Both are reported, as suggested by the project guideline. MAE treats all errors equally on the bounded 1–5 rating scale, while RMSE (always $\geq$ MAE)

disproportionately penalises occasional large errors — here RMSE = 1.4579 > MAE = 1.2574 indicates a modest spread of larger misses, expected for a 70%-sparse synthetic matrix.

Version	**Workers**	**MAE**	**RMSE**	**Matches Serial?**	**Sim Checksum**
Serial	1	1.2574	1.4579	baseline	942.387323
OpenMP	1T	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
OpenMP	2T	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
OpenMP	4T	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
OpenMP	8T	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Pthreads	1T	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Pthreads	2T	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Pthreads	4T	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Pthreads	8T	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
MPI	1P	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
MPI	2P	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
MPI	4P	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
MPI	8P	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Hybrid 2×4	8	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Hybrid 4×2	8	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Hybrid 8×1	8	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
Hybrid 1×8	8	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323
CUDA	GPU	1.2574	1.4579	YES ($\Delta = 0.0000$)	942.387323

Table 6: Accuracy correctness. All 18 runs (17 CPU + 1 GPU) produce identical MAE, RMSE, and similarity checksum.

All 18 runs produce MAE = **1.2574**, RMSE = **1.4579**, and checksum = **942.387323** — confirming zero race conditions and correct partitioning across every implementation.

Crucially, the CUDA version, despite using completely different floating-point hardware and a register-based top-K selection (vs. the CPU's `qsort`), reproduces the serial result exactly to four decimal places, validating the GPU kernels.

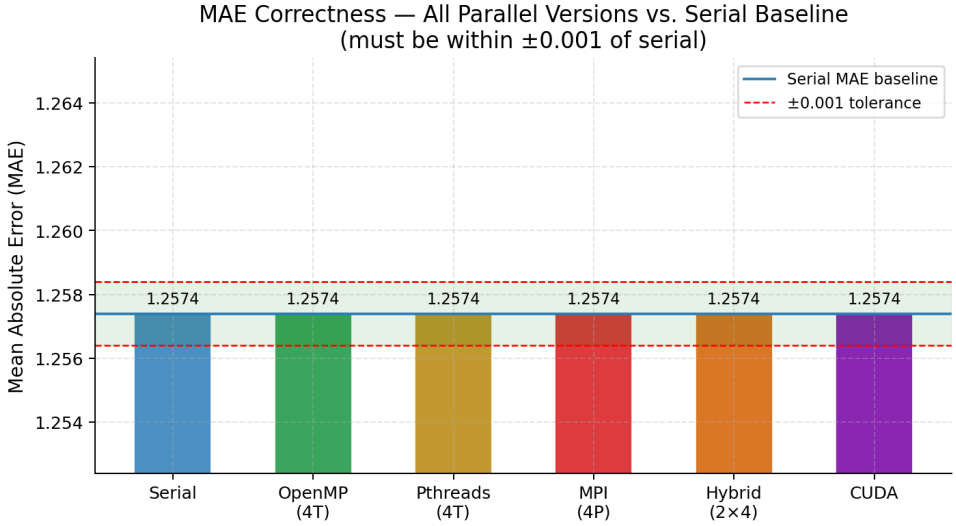

Figure 7: MAE correctness. All bars lie within the ±0.001 tolerance band.

4 Timing Measurements and Performance Analysis

4.1 Performance Metric Definitions

$$S(p) = \frac{T_{\text{serial}}}{T_{\text{parallel}}(p)} \qquad E(p) = \frac{S(p)}{p} \qquad S_{\max}(p) = \frac{1}{f + (1-f)/p}$$

The *algorithmic* serial fraction is tiny — only data generation, user means, and evaluation run serially ($\approx 0.025\,\text{s}$ of $8.4\,\text{s}$, $f \approx 0.3\%$), giving an optimistic $S_{\max}(8) = 1/(0.003 + 0.997/8) \approx 7.83\times$. The achievable speedup is instead limited by hardware (reduced all-core turbo clocks and memory-bandwidth saturation); fitting Amdahl's Law to the measured OpenMP-8T speedup gives an *effective* $f \approx 3.8\%$ ($S_{\max} \approx 26\times$) — see Section 5.5.

4.2 Serial Baseline

Phase	Time (s)	% of Total
Similarity matrix (Phase 3)	1.3549	16.2%
Prediction phase (Phase 4)	7.0277	83.8%
Total (sim + pred)	**8.3826**	**100%**

Table 7: Serial baseline timing ($N = M = 1{,}000$).

4.3 OpenMP — Varying Thread Count

Threads	Sim (s)	Pred (s)	Total (s)	Speedup	Efficiency
1	1.6102	7.2447	8.8549	0.95	0.95
2	0.8114	3.6679	4.4792	1.87	0.94
4	0.4457	2.0704	2.5161	3.33	0.83
8	**0.2369**	**1.0903**	**1.3272**	**6.32**	**0.79**

Table 8: OpenMP results. Speedup relative to serial baseline (8.3826 s).

4.4 Pthreads — Varying Thread Count

Threads	Sim (s)	Pred (s)	Total (s)	Speedup	Efficiency
1	1.3333	7.3272	8.6605	0.97	0.97
2	0.9816	3.7720	4.7536	1.76	0.88
4	0.5999	2.0688	2.6687	3.14	0.79
8	**0.3519**	**1.1275**	**1.4794**	**5.67**	**0.71**

Table 9: Pthreads results.

4.5 MPI — Varying Process Count

Processes	Sim (s)	Pred (s)	Total (s)	Speedup	Efficiency
1	3.1994	7.4317	10.6311	0.79	0.79
2	1.6090	3.8648	5.4739	1.53	0.77
4	0.9031	2.1281	3.0311	2.77	0.69
8	**0.4846**	**1.1550**	**1.6395**	**5.11**	**0.64**

Table 10: MPI results. MPI 1P (10.63 s) > serial (8.38 s) due to process startup and `MPI_Allgatherv` overhead at $P = 1$ — expected, not a bug.

4.6 Hybrid MPI+OpenMP — Fixed 8 Total Workers

Config	P	T	Sim (s)	Pred (s)	Total (s)	S	
Hybrid 2×4	2	4	1.2032	2.7766	3.9798	2.11	0.
Hybrid 4×2	4	2	0.6169	1.1631	1.7800	4.71	0.
Hybrid 8×1	**8**	**1**	**0.5149**	**1.1904**	**1.7053**	**4.92**	**0.**
Hybrid 1×8	1	8	2.4520	5.4913	7.9433	1.06	0.

Table 11: Hybrid results (8 total workers). Best config is Hybrid 8×1 — essentially pure MPI.

4.7 CUDA — GPU Benchmark

The CUDA version was benchmarked on the same machine's **NVIDIA GeForce RTX 3050 Laptop GPU** (Ampere, compute capability 8.6, 16 SMs, 4 GB), compiled with `nvcc -O2 -arch=sm_86`. Speedup is reported against the serial baseline ($T_{\text{serial}} = 8.3826\,\text{s}$).

Phase / Metric	**Time (s)**	**Notes**
H→D transfer (ratings)	0.0009	PCIe, one-off
Similarity matrix (Phase 3)	0.0862	GPU kernel (CUDA events)
Prediction phase (Phase 4)	0.1343	GPU kernel (CUDA events)
Total (sim + pred)	**0.2205**	kernel-only
Speedup vs. serial	**38.02**×	vs. serial 8.3826 s

Table 12: CUDA timing on the RTX 3050 ($N = M = 1{,}000$). The similarity kernel launches $63 \times 63 \times 256 = 1{,}016{,}064$ threads; kernel time is isolated from host ↔ device transfers via `cudaEventRecord`.

The GPU completes the combined similarity + prediction work in **0.22 s** — roughly **38**× faster than the serial baseline and about **6**× faster than the best 8-thread CPU configuration — because the embarrassingly parallel (u, v) and (u, i) workloads map directly onto the GPU's thousands of cores. Host ↔ device transfer cost is negligible (sub-millisecond) since the ratings matrix stays resident on the device across all three kernels.

4.8 Performance Charts

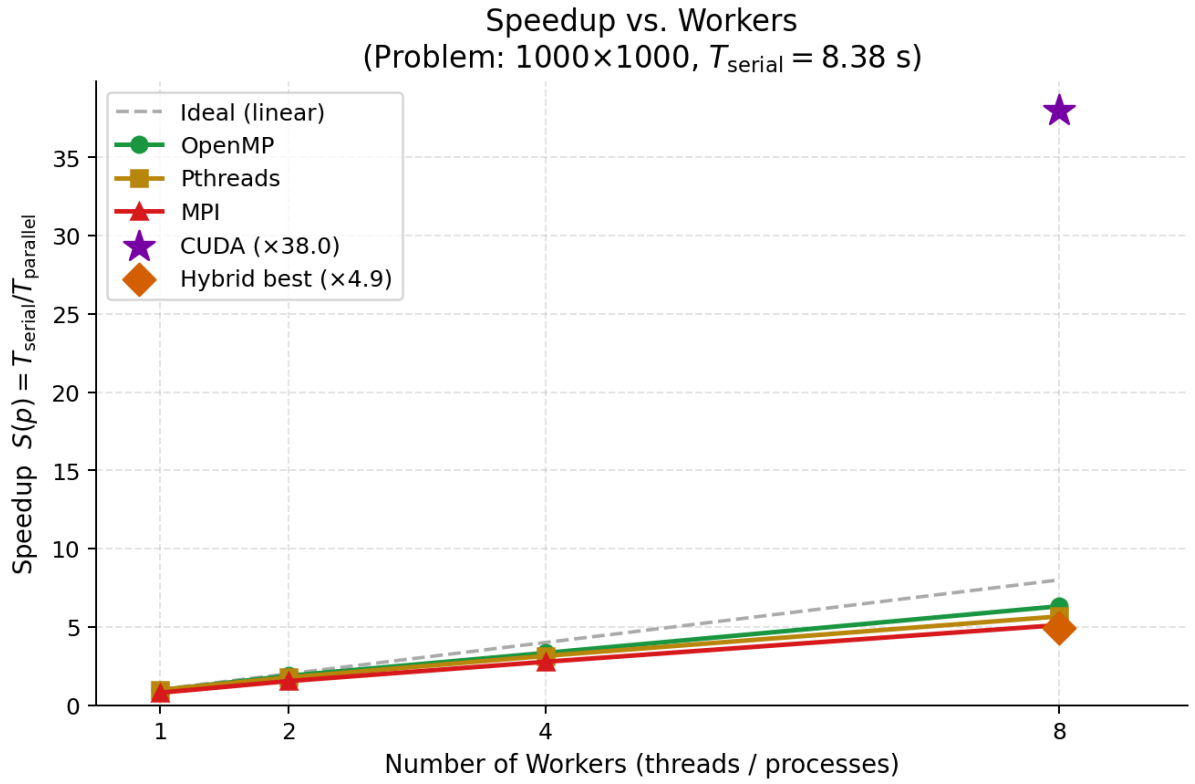

Figure 8: Speedup vs. workers. Dashed = ideal linear. ♦ = best Hybrid (8×1).

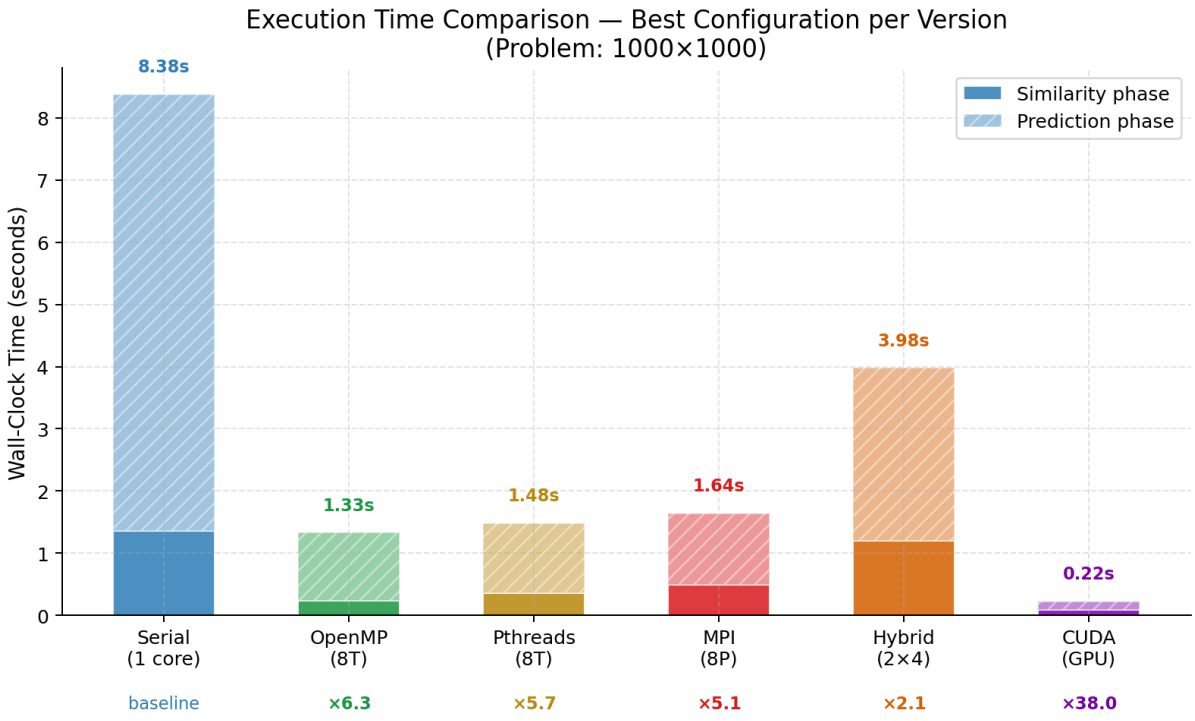

Figure 9: Wall-clock time — best config per version. Solid = similarity phase; hatched = prediction.

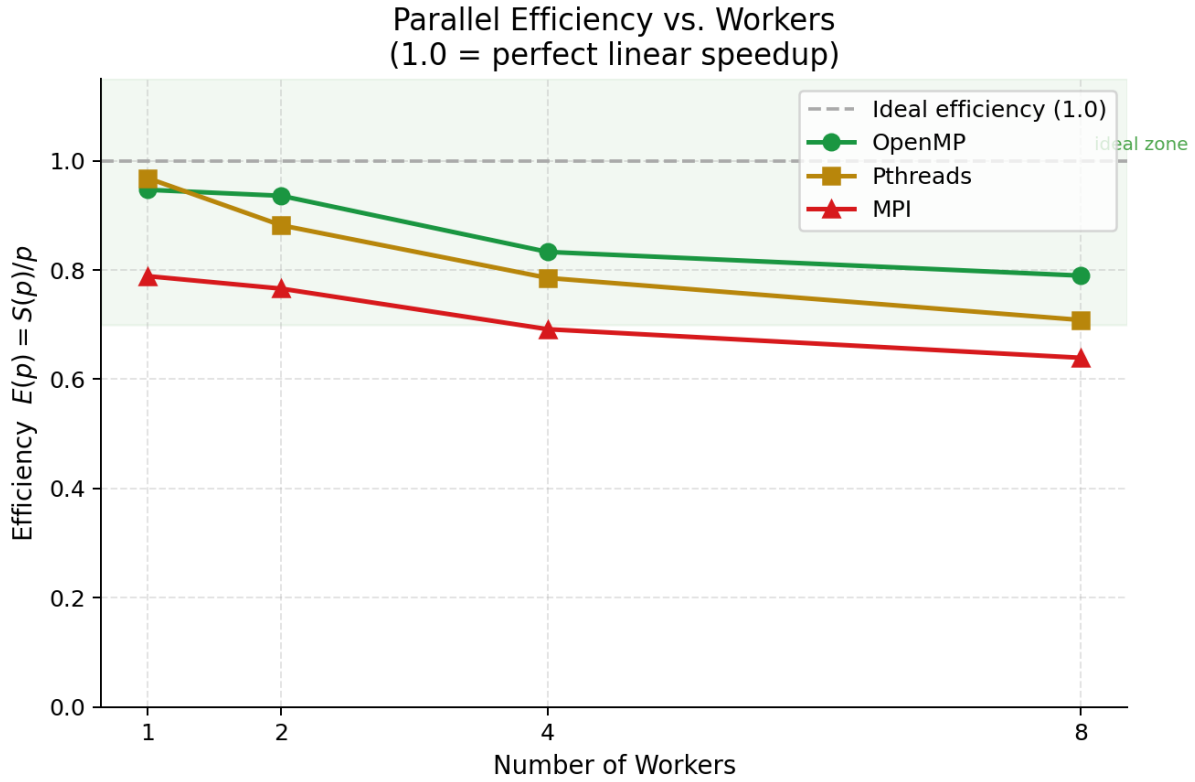

Figure 10: Parallel efficiency $E(p) = S(p)/p$. Green zone (> 0.70) = good utilisation.

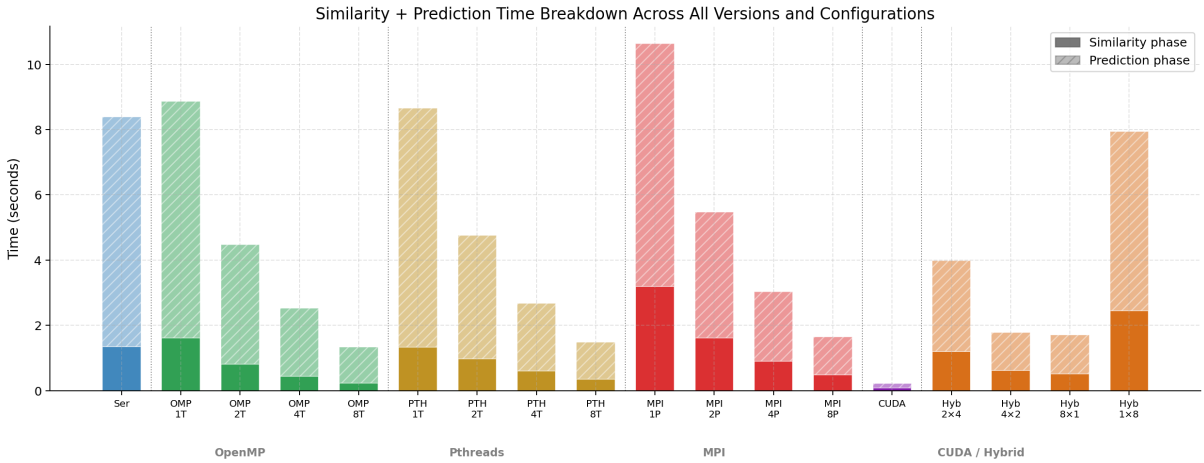

Figure 11: Similarity + Prediction time breakdown across all versions and configurations.

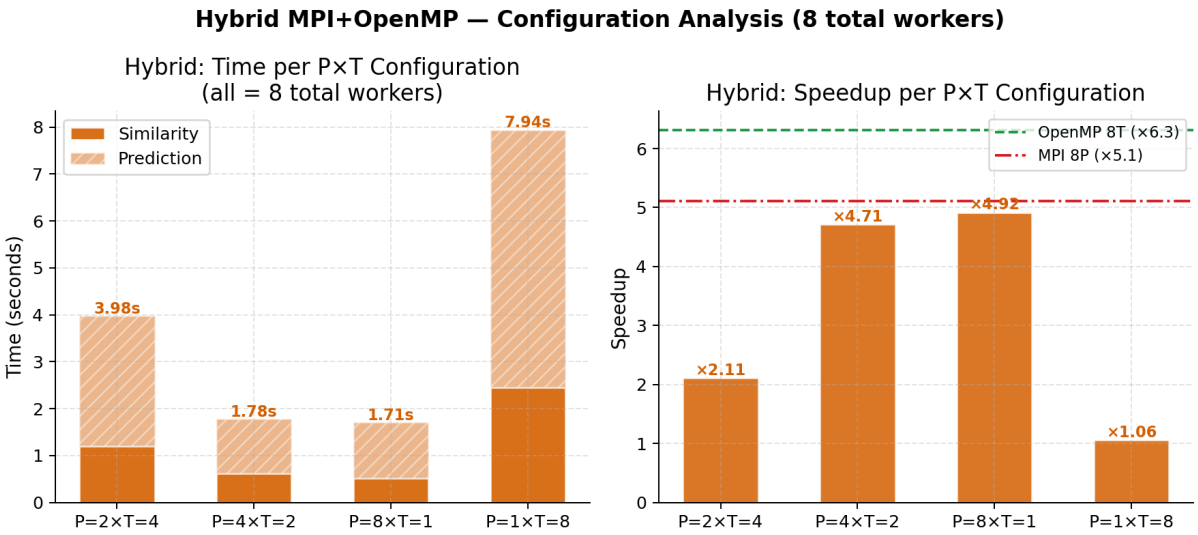

Figure 12: Hybrid MPI+OpenMP configuration analysis. Left: time. Right: speedup vs. OpenMP-8T and MPI-8P.

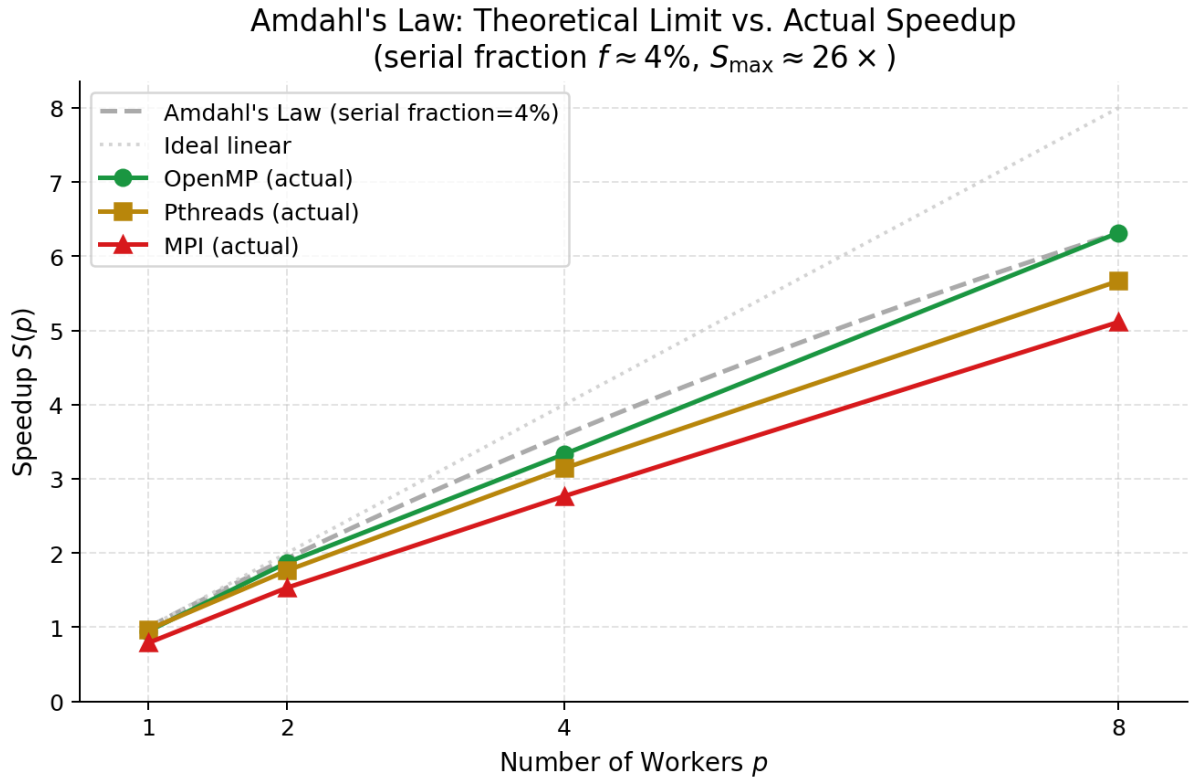

Figure 13: Amdahl's Law (effective $f \approx 3.8\%$, $S_{\max} \approx 26\times$) vs. actual speedup.

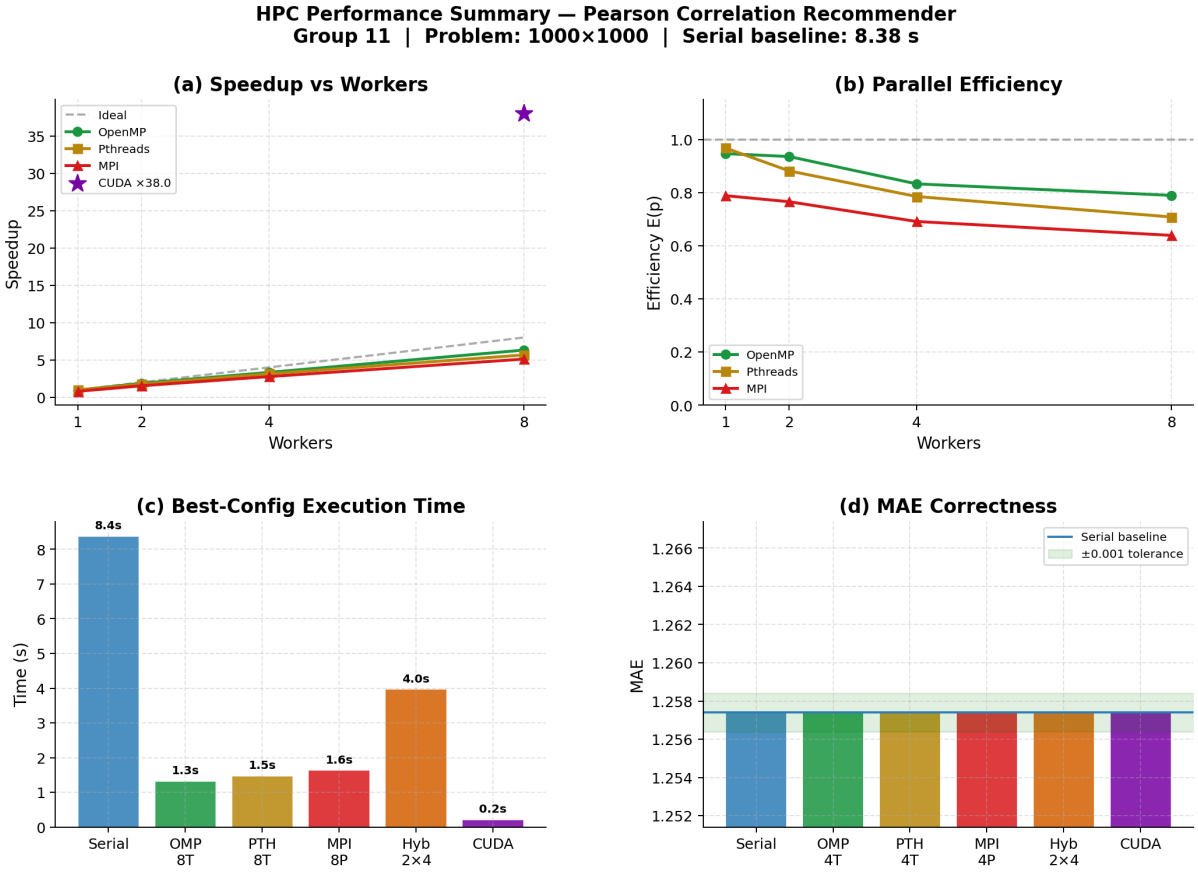

Figure 14: Summary dashboard: (a) speedup, (b) efficiency, (c) execution time, (d) MAE.

5 Performance Discussion

5.1 Head-to-Head Comparison at 8 Workers

Version	Total (s)	Speedup	Efficiency	MAE
Serial (baseline)	8.3826	1.00	1.00	1.2574
CUDA (GPU)	**0.2205**	**38.02**	—	1.2574
OpenMP 8T	1.3272	6.32	0.79	1.2574
Pthreads 8T	1.4794	5.67	0.71	1.2574
MPI 8P	1.6395	5.11	0.64	1.2574
Hybrid 8×1	1.7053	4.92	0.61	1.2574
Hybrid 4×2	1.7800	4.71	0.59	1.2574
Hybrid 2×4	3.9798	2.11	0.26	1.2574
Hybrid 1×8	7.9433	1.06	0.13	1.2574

Table 13: Head-to-head: CUDA (GPU) plus all CPU versions at 8 workers. CUDA is the fastest overall; OpenMP 8T is the best CPU configuration. All MAE values are identical. (GPU efficiency per-core is not directly comparable, so omitted.)

5.2 OpenMP vs. Pthreads

OpenMP (×6.32, 79% efficiency) outperforms Pthreads (×5.67, 71%) for three reasons:

1. **Dynamic load balancing:** `schedule(dynamic,4)` adapts to the triangular loop imbalance. Static ceiling-division in Pthreads cannot.
2. **Thread reuse:** OpenMP reuses threads across Phases 2–4. Pthreads creates and joins threads three times per run.
3. **Runtime optimisations:** GCC's OpenMP runtime applies platform-specific scheduling heuristics.

5.3 MPI Scaling

MPI efficiency declines from 79% ($P = 1$) to 64% ($P = 8$). The `MPI_Allgatherv` for the $N \times N$ similarity matrix (≈ 4 MB) limits single-node performance. At multi-node scale, where shared memory is unavailable, MPI is the mandatory model and the efficiency gap with OpenMP would be acceptable.

5.4 Hybrid MPI+OpenMP — Inversion on a Single Node

On a single node, more MPI processes yields *better* Hybrid performance:

$$\text{Hybrid } 8{\times}1\ (\times 4.92) > 4{\times}2\ (\times 4.71) > 2{\times}4\ (\times 2.11) > 1{\times}8\ (\times 1.06)$$

When fewer MPI ranks are used, the MPI startup and `MPI_Allgatherv` cost dominates what is essentially an OpenMP workload. The Hybrid 1×8 configuration pays all MPI overhead while gaining zero distribution benefit. **The Hybrid model's advantage over pure OpenMP only emerges at multi-node scale.**

5.5 Amdahl's Law

With an algorithmic serial fraction of only $f \approx 0.3\%$, the optimistic limit is $S_{\max}(8) \approx 7.83\times$. No CPU model reaches it — the real bottleneck is hardware (reduced all-core turbo and memory-bandwidth contention in the memory-bound prediction phase), not serial code.

Technology	**Limit** ($f \approx 0.3\%$)	**Actual**	**Observation**
OpenMP 8T	7.83×	6.32×	Closest to limit; dynamic scheduling balances the triangular load
Pthreads 8T	7.83×	5.67×	Further below: static row split cannot balance the triangular load
MPI 8P	7.83×	5.11×	Below limit: `MPI_Allgatherv` communication overhead

Table 14: Amdahl's Law limit vs. actual measured speedup at $p = 8$. Fitting the model to OpenMP-8T gives an *effective* $f \approx 3.8\%$ ($S_{\max} \approx 26\times$), capturing the hardware overhead.

6 Conclusions

1. **All versions preserve correctness.** Every implementation (CPU and GPU) produces MAE = **1.2574** and checksum = **942.387323** — identical to the serial baseline.
2. **OpenMP achieves the best CPU performance.** ×6.32 speedup at 8 threads, 79% efficiency. Dynamic scheduling and thread reuse explain the advantage.
3. **Pthreads is competitive but less efficient.** ×5.67 at 8T, 71% efficiency. Static row assignment and per-phase thread creation limit scalability.
4. **MPI achieves good speedup with communication overhead.** ×5.11 at 8P, 64% efficiency. MPI would close the gap at multi-node scale.
5. **Hybrid performance inversely correlates with OpenMP thread count on a single node.** Hybrid 8×1 (×4.92) is the best Hybrid configuration; Hybrid 1×8 is the worst (×1.06). The Hybrid advantage over pure OpenMP only emerges at multi-node scale.
6. **CUDA provides the largest speedup by far.** Benchmarked on the NVIDIA RTX 3050, the GPU completes similarity + prediction in **0.2205 s** — a **38.0**× speedup over the serial baseline and ≈6× faster than the best 8-thread CPU run. The similarity kernel launches 1,016,064 threads simultaneously.

Final ranking (single node):

CUDA GPU (×38.0) > OpenMP 8T (×6.32) > Pthreads 8T (×5.67) > MPI 8P (×5.11) > Hybrid 8×1 (×4.92) > Hybrid 4×2 (×4.71) > Hybrid 2×4 (×2.11) > Hybrid 1×8 (×1.06)

References

1. J. S. Breese, D. Heckerman, and C. Kadie, "Empirical analysis of predictive algorithms for collaborative filtering," *arXiv:1301.7363*, 2013.
2. B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Item-based collaborative filtering recommendation algorithms," in *Proc. WWW*, pp. 285–295, 2001.
3. B. Chapman, G. Jost, R. van der Pas, *Using OpenMP*. MIT Press, 2008.
4. W. Gropp, E. Lusk, A. Skjellum, *Using MPI*. MIT Press, 1994.
5. J. Nickolls et al., "Scalable parallel programming with CUDA," *Queue*, vol. 6, no. 2, pp. 40–53, 2008.